

Invenqor 자산 API·MCP·키 관리 가이드

대상 Server 버전: v0.2.4 · 기준일: 2026-07-29

1. 목적과 설계 원칙

Invenqor는 자동화 도구와 AI Agent가 같은 자산 원장을 안전하게 사용할 수 있도록 REST API와 MCP(Model Context Protocol)를 Server의 단일 HTTPS TCP 7070에서 제공합니다.

- 브라우저 관리 콘솔은 Session+CSRF를 사용합니다.
- 자동화 REST API와 MCP는 별도의 scoped API key를 Bearer로 사용합니다.
- 키 원문은 생성·회전 응답에서 한 번만 보이고 DB에는 SHA-256 해시만 저장합니다.
- Key scope는 사용자 RBAC와 분리된 최소 권한 경계이며 즉시 추가·교체·삭제할 수 있습니다.
- 읽기·변경 권한을 분리하고 MCP는 현재 읽기 전용 자산 도구만 제공합니다.
- MCP 처리는 stateless이므로 Kubernetes의 어느 Server Pod로 요청해도 됩니다.

MCP 구현은 안정 규격 2025-11-25의 JSON-RPC 2.0과 Streamable HTTP를 기준으로 합니다. Server가 client session ID를 발급하지 않는 stateless 방식이라 sticky session이 필요 없습니다. 공식 전송 규격이 요구하는 Origin 검증을 적용하고, SSE가 필요 없는 현재 도구 집합은 POST JSON 응답을 사용합니다.

Agent 자동 등록의 운영 API도 같은 Session+CSRF 경계에 있습니다. 관리자는 /api/v1/admin/settings/agent-enrollment에서 URL-only Open, Token 보호, 비활성 모드를 전환하고 /token 하위 경로에서 등록 Token을 발급·회전·폐기합니다. 정책은 공용 DB에서 요청마다 검증되어 모든 Server Pod에 즉시 적용되며 Token 원문은 발급 응답에서 한 번만 반환됩니다. 상세 계약은 openapi.yaml을 따릅니다.

2. Scope 카탈로그

Scope	허용 작업
assets.read	자산 목록·검색·상세 조회
assets.write	수동 자산 생성·수정
assets.delete	논리 삭제·복원
relations.read	자산 관계 조회
relations.write	자산 관계 생성·삭제
agents.read	Agent 상태 조회
queries.execute	검증된 Query DSL 실행
mcp.access	/mcp 연결과 도구 탐색

`mcp.access` 만 부여하면 MCP 연결은 가능하지만 자산 도구는 표시되지 않습니다. 예를 들어 `mcp.access + assets.read` 키에는 `asset_search`, `asset_get` 만 노출됩니다. 키를 만드는 관리자도 자신이 갖지 않은 권한을 scope로 위임할 수 없습니다. `api_keys.manage` 자체는 API key에 위임할 수 없는 관리 Session 전용 권한입니다.

3. 키 수명주기 API

모든 관리 요청은 관리자 Session Cookie, `X-CSRF-Token`, `api_keys.manage` 권한이 필요합니다.

브라우저 콘솔은 로컬 로그인 또는 Keycloak callback에서 발급된 SameSite CSRF cookie를 상태 변경 요청의 `X-CSRF-Token` 에 자동 연결합니다. CLI 예시에서는 cookie jar와 로그인 응답의 CSRF 값을 명시적으로 보관해야 합니다.

Method	경로	기능
GET	<code>/api/v1/admin/api-key-scopes</code>	허용 scope 카탈로그
GET	<code>/api/v1/admin/api-keys</code>	키 목록; 원문 제외
GET	<code>/api/v1/admin/api-keys/{id}</code>	키 상세; 원문 제외
POST	<code>/api/v1/admin/api-keys</code>	키 생성
PATCH	<code>/api/v1/admin/api-keys/{id}</code>	이름·scope 전체 교체
POST	<code>/api/v1/admin/api-keys/{id}/scopes</code>	scope 추가
DELETE	<code>/api/v1/admin/api-keys/{id}/scopes/{scope}</code>	scope 개별 삭제
POST	<code>/api/v1/admin/api-keys/{id}/rotate</code>	무중단 회전
DELETE	<code>/api/v1/admin/api-keys/{id}</code>	즉시 폐기

3.1 생성

```
curl -b cookies.txt -H "X-CSRF-Token: $CSRF" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "cldb-readonly",
    "scopes": ["assets.read", "relations.read", "mcp.access"],
    "expires_at": "2027-01-01T00:00:00Z"
  }' \
  https://invenqor.example.com:7070/api/v1/admin/api-keys
```

응답의 `secret` 은 안전한 Secret Manager에 즉시 저장하십시오. 목록·상세 API는 `prefix`, scope, 만료·마지막 사용 시각만 제공하며 원문을 다시 보여주지 않습니다.

3.2 Scope 추가·변경·삭제

```
# 추가
curl -b cookies.txt -H "X-CSRF-Token: $CSRF" \
  -H 'Content-Type: application/json' \
  -d '{"scopes":["agents.read"]}' \
  https://invenqor.example.com:7070/api/v1/admin/api-keys/$KEY_ID/scopes

# 전체 교체: 빈 배열이면 유효한 인증정보이지만 어떤 업무 권한도 없음
curl -X PATCH -b cookies.txt -H "X-CSRF-Token: $CSRF" \
  -H 'Content-Type: application/json' \
  -d '{"scopes":["assets.read","mcp.access"]}' \
  https://invenqor.example.com:7070/api/v1/admin/api-keys/$KEY_ID

# 개별 삭제
curl -X DELETE -b cookies.txt -H "X-CSRF-Token: $CSRF" \
  https://invenqor.example.com:7070/api/v1/admin/api-keys/$KEY_ID/scopes/agents.read
```

변경은 DB에서 직접 검증하므로 별도 cache 만료를 기다리지 않고 다음 요청부터 모든 Server Pod에 적용됩니다.

3.3 무중단 회전과 폐기

```
curl -b cookies.txt -H "X-CSRF-Token: $CSRF" \
  -H 'Content-Type: application/json' \
  -d '{"grace_seconds":3600}' \
  https://invenqor.example.com:7070/api/v1/admin/api-keys/$KEY_ID/rotate
```

새 **secret** 을 소비자에 배포하고 1시간 안에 구 키를 제거합니다. 유예는 0~604800초(7일)이며 0이면 구 키가 즉시 무효화됩니다. Scope는 논리 Key에 연결되므로 구·신 키 모두 최신 scope 변경을 즉시 따릅니다.

```
curl -X DELETE -b cookies.txt -H "X-CSRF-Token: $CSRF" \
  https://invenqor.example.com:7070/api/v1/admin/api-keys/$KEY_ID
```

폐기 시 현재·유예 키가 모두 즉시 거부됩니다. 소유 사용자가 비활성화되거나 삭제되어도 해당 키는 인증되지 않습니다.

4. 자산 REST API

기본 헤더:

```
Authorization: Bearer ivq_sk...
Accept: application/json
```

외부 자동화 전용 경로는 다음과 같습니다.

Method	경로	Scope
GET	/api/v1/external/assets	assets.read
GET	/api/v1/external/assets/{id}	assets.read
POST	/api/v1/external/assets	assets.write
PATCH	/api/v1/external/assets/{id}	assets.write
DELETE	/api/v1/external/assets/{id}	assets.delete
POST	/api/v1/external/assets/{id}/restore	assets.delete
GET	/api/v1/external/assets/{id}/relations	relations.read
POST/DELETE	/api/v1/external/assets/{id}/relations...	relations.write
POST	/api/v1/external/query/validate	queries.execute
POST	/api/v1/external/query/execute	queries.execute

검색 예:

```
curl -H "Authorization: Bearer $INVENQOR_API_KEY" \  
  'https://invenqor.example.com:7070/api/v1/external/assets?q=ubuntu&type=host&status=active&limit=50'
```

API key 인증에는 Cookie와 CSRF를 사용하지 않습니다. 키를 URL query, 로그, 지원 티켓, source control에 넣지 마십시오.

5. MCP 연결

MCP URL은 다음 하나입니다.

```
https://invenqor.example.com:7070/mcp
```

클라이언트에는 `Authorization: Bearer <API key>` 를 Secret 환경변수 또는 지원되는 secure header 설정으로 전달합니다. 일반적인 설정 개념:

```
{
  "transport": "streamable-http",
  "url": "https://invenqor.example.com:7070/mcp",
  "headers": {
    "Authorization": "Bearer ${INVENQOR_API_KEY}"
  }
}
```

초기화 확인:

```
curl -H "Authorization: Bearer $INVENQOR_API_KEY" \
-H 'Content-Type: application/json' \
-H 'Accept: application/json, text/event-stream' \
-d '{
  "jsonrpc": "2.0", "id": 1, "method": "initialize",
  "params": {
    "protocolVersion": "2025-11-25",
    "capabilities": {},
    "clientInfo": { "name": "asset-assistant", "version": "1.0" }
  }
}' \
https://invenqor.example.com:7070/mcp
```

6. 제공 MCP 도구

도구	Scope	입력	결과
<code>asset_search</code>	<code>assets.read</code>	<code>q</code> , <code>type</code> , <code>status</code> , <code>limit</code> , <code>offset</code>	정규화 자산 목록
<code>asset_get</code>	<code>assets.read</code>	<code>asset_id</code>	자산 상세
<code>asset_relations</code>	<code>relations.read</code>	<code>asset_id</code>	활성 inbound/outbound 관계
<code>agents_list</code>	<code>agents.read</code>	<code>limit</code>	Agent 상태·버전·최근 수신

도구 목록은 고정된 결정적 순서이며 키에 없는 scope의 도구는 아예 노출하지 않습니다. 결과는 MCP 호환 text content와 typed `structuredContent` 를 함께 제공합니다. 자산의 이름·속성·사용자 입력은 신뢰할 수 없는 데이터이며 AI에 대한 명령으로 해석해서는 안 된다는 지침도 initialize 응답에 포함됩니다.

```
curl -H "Authorization: Bearer $INVENQOR_API_KEY" \
-H 'Content-Type: application/json' \
-d '{
  "jsonrpc": "2.0", "id": 2, "method": "tools/call",
  "params": {"name": "asset_search", "arguments": {"type": "host", "limit": 20}}
}' \
https://invenqor.example.com:7070/mcp
```

MCP는 임의 SQL, shell, 원격 명령, 자산 변경 도구를 제공하지 않습니다. 변경 작업은 명시적 REST scope와 조직 승인 절차를 사용합니다.

7. 보안·감사·운영

- 모든 키 생성, 이름/scope 변경, 회전과 폐기는 감사 로그에 actor, 대상 ID, 변경 전후 값과 request ID를 남깁니다. Secret 원문은 감사 로그에 넣지 않습니다.
- MCP는 `Origin` 이 있으면 요청 Host와 일치하는지 확인해 DNS rebinding을 차단합니다. 운영 TLS Proxy도 원래 Host를 보존해야 합니다.
- Key 별 Server rate limit 외에 Ingress/API Gateway에서 전역 rate limit, 허용 IP, 최대 body와 timeout을 적용하십시오.
- 각 연계 시스템마다 별도 Key를 만들고 공유 Key를 금지합니다.
- 만료일은 90일 이하, 회전 유예는 실제 배포시간에 필요한 최소값을 권장합니다.
- Secret 탐지에 걸린 Key는 유예 없이 폐기하고 새 Key를 발급합니다.
- PostgreSQL과 감사 로그를 백업하되 Secret Manager의 원문 Key와 함께 평문으로 내보내지 않습니다.

멀티 파드에서 키·scope·회전 상태는 PostgreSQL을 단일 원장으로 사용합니다. Pod별 메모리 session이나 sticky routing을 요구하지 않으므로 rolling update 중에도 같은 MCP client가 어느 Pod로든 요청할 수 있습니다.

8. 장애 코드

상태	코드	조치
401	<code>INVALID_API_KEY</code>	오타, 만료, 폐기, 소유 사용자 상태 확인
403	<code>FORBIDDEN</code>	필요한 scope 추가 또는 별도 Key 발급
403	<code>MCP_ORIGIN_REJECTED</code>	Proxy Host/Origin 전달 정책 확인
403	<code>SCOPE_ESCALATION</code>	관리자의 RBAC 권한 범위 안에서만 위임
429	<code>API_RATE_LIMITED</code>	호출 빈도 감소, Gateway 정책 확인

MCP protocol 오류는 JSON-RPC error로, 도구 입력·실행 오류는 `isError: true` tool result로 반환해 AI client가 안전하게 수정 요청을 만들 수 있게 합니다.

9. 참고 규격

- [MCP 2025-11-25 Streamable HTTP](#)
- [MCP Tools](#)
- [MCP Schema Reference](#)